

综合实践报告

课程: 机器学习 实验名称: 影评数据分析与电影推荐 日期:4.1

姓名:雷文豪 学号:138022024031 专业班级:人工智能

1 实验目的

1. 通过认识基本的协同过滤算法，加深对 k -近邻的理解，并分析电影评分数据集求解应用问题。
2. 学习解题设计、通过代码编写与运行实施解题、完成程序正确性的验证。

2 实验内容提要 with 实验设备

1. 针对待解问题设计适合的求解方法。
2. 对给定数据集，设计求解过程、编写程序实施求解。
3. 实验结果的分析与确认。
4. 实验设备：个人电脑或便携式电脑，并安装 Windows 等操作系统和编程开发环境。

3 影评数据分析与电影推荐任务

✓ 填写要求：程序输出数字文字时只截图有数字文字的部分（小截图）；若插入文档的截图仍比较大，将截图拉小点，再进行一次截图；代码使用无修饰、单倍间距、黑色新罗马体。

3.1 目的

对影评数据集，利用机器学习方法进行影评数据分析，并结合分析结果，实现给用户推荐电影。该任务可以作为分类问题进行求解。数据集文件如下：

- `movies.csv`: 包含电影 ID、电影名称。
- `ratings.csv`: 包含加密的用户 ID、电影 ID、评分值。

3.2 电影推荐的原理介绍

电影的推荐算法有很多，这里我们采用一种经典的方法：协同过滤算法，它又可以细分为两种：

- **基于用户的协同过滤方法：** 用于给用户 A 推荐有着相似观影兴趣的用户 B 喜欢观看的电影。
- **基于物品的协同过滤方法：** 根据用户过往看过的电影，从电影库中查找相似的电影进行推荐。

在协同过滤算法中，准确度量用户之间或物品之间的相似度是生成高质量推荐的基础，其度量方式也有许多种，这里我们主要用以下两种：

- **余弦相似度：** 通过测量两个非零向量在多维空间中夹角的余弦值来评估它们的相似度。给定两个 N 维向量 $\vec{A}$ 和 $\vec{B}$ ，它们之间的余弦相似度 $\text{sim}(\vec{A}, \vec{B})$ 定义为它们的点积除以它们模长的乘积：

$$\text{sim}(\vec{A}, \vec{B}) = \cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{\|\vec{A}\| \|\vec{B}\|}.$$

如果我们要计算用户 u 和用户 v 的相似度，向量的每个维度代表一个物品，维度的值代表用户对该物品的评分。设 I 为用户 u 和 v 共同评分过的物品集合， $r_{u,i}$ 表示用户 u 对物品 i 的评分，则公式可调整为：

$$\text{sim}(u, v) = \frac{\sum_{i \in I} r_{u,i} \cdot r_{v,i}}{\sqrt{\sum_{i \in I} r_{u,i}^2} \sqrt{\sum_{i \in I} r_{v,i}^2}}.$$

余弦相似度的取值范围是 $[-1, 1]$ 。当两个向量方向完全一致时，相似度为 1；正交时为 0；方向完全相反时为 -1。

- **Jaccard 相似度：** 主要用于比较有限样本集合的相似性与差异性。它不关注具体的评分数值，而是基于集合的交集和并集进行计算。给定两个集合 A 和 B ，Jaccard 相似度 $J(A, B)$ 定义为两个集合交集的元素个数除以并集的元素个数：

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}.$$

假设我们要计算用户 u 和用户 v 的相似度。设 I_u 为用户 u 交互过的物品集合， I_v 为用户 v 交互过的物品集合。则他们之间的 Jaccard 相似度可以表示为：

$$\text{sim}(u, v) = \frac{|I_u \cap I_v|}{|I_u \cup I_v|}.$$

Jaccard 相似度的取值范围严格在 $[0, 1]$ 之间。当两个集合完全相同时，相似度为 1；当两个集合没有任何交集时，相似度为 0。

3.3 解答参考步骤

1. **任务与目标。** 要解决这个任务，分为总目标与阶段性目标。

- 总目标：分析许多用户对许多电影的评分情况，给某用户推荐电影。
- 分目标 1：找出与指定用户有类似观影喜好的用户们；
- 分目标 2：找出与指定电影有相似观众群的那些电影；
- 分目标 3：利用 k -近邻分类方法，给某用户推荐相同喜好用户们所喜欢的电影。

2. 数据集的分析理解. 对于当前的数据, 可分别从用户和电影两个角度来分析。例如, 从用户角度, 可以看每个用户评了多少次, 评分的平均次数是多少; 评分的分布又是如何的等等。再比如, 从电角度, 电影被评分的次数是多少, 电影分数的分布又是如何的等等。

3. 电影推荐的流程设计. 注意, 我们需要根据当前的数据给用户推荐他们没有看过的, 但是很可能会喜欢的电影。大致的流程为:

- (a) 根据相似度建立关于用户 (或者电影) 的近邻矩阵;
- (b) 找到近邻的 k 个值;
- (c) 获取推荐的电影列表;
- (d) 去除用户已经看过的电影;
- (f) 输出推荐结果, 并使用学过的评价指标来评估推荐结果的好坏。

4. 结果展示.

(1) 源代码

```
import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore') # 忽略多余警告

# 超参数配置
CONFIG = {
    "interest_threshold": 4, # 兴趣阈值: 评分 $\geq$ 该值视为用户喜欢的电影
    "top_k": 10, # K近邻数量
    "alpha": 5, # 自适应加权的平衡参数
    "shrinkage_factor": 10, # Shrinkage收缩系数
    "top_n": 10, # 最终推荐的电影数量
    "random_state": 42 # 随机种子
}

# 数据加载与预处理模块
class DataProcessor:
    def __init__(self, ratings_path, movies_path):
        self.ratings_path = ratings_path
        self.movies_path = movies_path
        self.rating_matrix = None # 用户-电影评分矩阵 (n_users, n_movies)
        self.user_mean_rating = None # 每个用户的平均评分 (n_users,)
        self.user_id_map = None # 原始用户ID  $\rightarrow$  矩阵索引
        self.movie_id_map = None # 原始电影ID  $\rightarrow$  矩阵索引
        self.id_movie_map = None # 矩阵索引  $\rightarrow$  电影ID+名称
        self.n_users = 0
        self.n_movies = 0

    def load_and_process(self):
        # 加载数据并完成预处理, 构建评分矩阵
        # 加载原始数据
        ratings_df = pd.read_csv(self.ratings_path, header=None, names=["user_id", "movie_id", "rating"])
        movies_df = pd.read_csv(self.movies_path, header=None, names=["movie_id", "movie_name"])
```

```

# 构建ID到连续索引的映射
# 用户ID映射
unique_users = ratings_df["user_id"].unique()
self.user_id_map = {user_id: idx for idx, user_id in enumerate(unique_users)}
# 电影ID映射
unique_movies = movies_df["movie_id"].unique()
self.movie_id_map = {movie_id: idx for idx, movie_id in enumerate(unique_movies)}
# 电影索引 → 信息映射
self.id_movie_map = {idx: (row["movie_id"], row["movie_name"]) for idx, row in
movies_df.set_index("movie_id").reindex(unique_movies).reset_index().iterrows()}
self.n_users = len(unique_users)
self.n_movies = len(unique_movies)
print(f"数据加载完成: 用户数={self.n_users}, 电影数={self.n_movies}, 有效评分记录
={len(ratings_df)}")

# 构建用户-电影评分矩阵, 未评分位置使用0填充
self.rating_matrix = np.zeros((self.n_users, self.n_movies), dtype=np.float32)
for _, row in ratings_df.iterrows():
    user_idx = self.user_id_map[row["user_id"]]
    movie_idx = self.movie_id_map.get(row["movie_id"], -1)
    if movie_idx != -1:
        self.rating_matrix[user_idx, movie_idx] = row["rating"]

# 计算每个用户的平均评分, 未评分电影按中位数填充
self.user_mean_rating = np.zeros(self.n_users, dtype=np.float32)
for user_idx in range(self.n_users):
    rated_mask = self.rating_matrix[user_idx] > 0
    if np.sum(rated_mask) > 0:
        self.user_mean_rating[user_idx] = np.mean(self.rating_matrix[user_idx, rated_mask])
    else:
        self.user_mean_rating[user_idx] = 3.0

return self

# 用户建模模块
class UserModeler:
    def __init__(self, data_processor, interest_threshold):
        self.data_processor = data_processor
        self.interest_threshold = interest_threshold
        self.rating_vectors = None    # 评分向量表示: 用户-电影评分矩阵
        self.interest_sets = None    # 兴趣集合表示: 每个用户喜欢的电影索引集合
        self.interest_matrix = None  # 兴趣二值矩阵 (n_users, n_movies), 喜欢为1, 否则0
        self.interest_sum = None     # 每个用户的喜欢电影数量 (n_users,)

    def build_user_representation(self):
        # 构建两种用户表示: 评分向量 + 兴趣集合
        # 评分向量表示
        self.rating_vectors = self.data_processor.rating_matrix

        # 兴趣集合与二值矩阵表示
        # 评分 ≥ 阈值且有评分的电影, 视为用户喜欢的电影
        self.interest_matrix = ((self.rating_vectors >= self.interest_threshold) & (self.rating_vectors >
0)).astype(np.int8)
        # 每个用户的兴趣集合
        self.interest_sets = [set(np.where(self.interest_matrix[user_idx] == 1)[0]) for user_idx in
range(self.data_processor.n_users)]
        # 每个用户的喜欢电影数量
        self.interest_sum = np.sum(self.interest_matrix, axis=1)

        print(f"用户建模完成: 平均每个用户喜欢{np.mean(self.interest_sum):.2f}部电影")

```

```
return self
```

```
# 相似度计算模块
```

```
class SimilarityCalculator:
```

```
    def __init__(self, user_modeler, alpha, shrinkage_factor):
        self.user_modeler = user_modeler
        self.alpha = alpha
        self.shrinkage_factor = shrinkage_factor
        self.n_users = user_modeler.data_processor.n_users
        self.c_matrix = None          # 两两用户共同评分数量矩阵 (n_users, n_users)
        self.cos_sim_matrix = None    # 共同子空间余弦相似度矩阵
        self.jac_sim_matrix = None    # 兴趣Jaccard相似度矩阵
        self.fused_sim_matrix = None  # 自适应加权融合后的相似度矩阵
        self.final_sim_matrix = None  # 经过Shrinkage处理的最终相似度矩阵
```

```
    def calc_common_rating_count(self):
        # 矩阵运算计算两两用户的共同评分数量（无循环，高效）
        # 评分二值化：有评分=1，无评分=0
        rating_binary = (self.user_modeler.rating_vectors > 0).astype(np.int8)
        # 矩阵相乘得到共同评分数量：c_matrix[u][v] = u和v共同评分的电影数
        self.c_matrix = rating_binary @ rating_binary.T
        return self
```

```
    def calc_cosine_similarity(self):
        # 计算共同评分子空间的余弦相似度
        self.cos_sim_matrix = np.zeros((self.n_users, self.n_users), dtype=np.float32)
        rating_matrix = self.user_modeler.rating_vectors
```

```
        for u in range(self.n_users):
            for v in range(u+1, self.n_users):
                # 共同评分的电影掩码
                common_mask = (rating_matrix[u] > 0) & (rating_matrix[v] > 0)
                common_count = np.sum(common_mask)
                if common_count == 0:
                    self.cos_sim_matrix[u, v] = 0.0
                    self.cos_sim_matrix[v, u] = 0.0
                    continue
```

```
                # 提取共同评分的向量
                u_rating = rating_matrix[u, common_mask]
                v_rating = rating_matrix[v, common_mask]
```

```
                # 计算余弦相似度
                dot_product = np.sum(u_rating * v_rating)
                norm_u = np.linalg.norm(u_rating)
                norm_v = np.linalg.norm(v_rating)
                norm_product = norm_u * norm_v
```

```
                if norm_product == 0:
                    cos_sim = 0.0
                else:
                    cos_sim = dot_product / norm_product
```

```
                self.cos_sim_matrix[u, v] = cos_sim
                self.cos_sim_matrix[v, u] = cos_sim
```

```
        # 自身相似度设为0
        np.fill_diagonal(self.cos_sim_matrix, 0.0)
        return self
```

```
    def calc_jaccard_similarity(self):
```

```

# 矩阵运算计算兴趣集合的Jaccard相似度
interest_matrix = self.user_modeler.interest_matrix
# 交集数量：两个用户都喜欢的电影数
intersection = interest_matrix @ interest_matrix.T
# 并集数量：|A ∪ B| = |A| + |B| - |A ∩ B|
union = np.expand_dims(self.user_modeler.interest_sum, 1) + self.user_modeler.interest_sum -
intersection
# 计算Jaccard，分母为0时设为0
with np.errstate(divide='ignore', invalid='ignore'):
    self.jac_sim_matrix = np.where(union == 0, 0.0, intersection / union)

# 自身相似度设为0
np.fill_diagonal(self.jac_sim_matrix, 0.0)
return self

def adaptive_weight_fusion(self):
# 自适应加权融合余弦相似度与Jaccard相似度
# 共同评分数量越多，余弦相似度权重越高；越少则Jaccard权重越高
with np.errstate(divide='ignore', invalid='ignore'):
    w_cos = self.c_matrix / (self.c_matrix + self.alpha)
    w_jac = self.alpha / (self.c_matrix + self.alpha)

# 融合相似度
self.fused_sim_matrix = w_cos * self.cos_sim_matrix + w_jac * self.jac_sim_matrix
return self

def shrinkage_process(self):
# Shrinkage收缩处理，降低样本不足时的相似度估计方差
with np.errstate(divide='ignore', invalid='ignore'):
    shrinkage_weight = self.c_matrix / (self.c_matrix + self.shrinkage_factor)
self.final_sim_matrix = shrinkage_weight * self.fused_sim_matrix
# 相似度范围限制在[-1,1]，保证数值稳定性
self.final_sim_matrix = np.clip(self.final_sim_matrix, -1.0, 1.0)
return self

def calculate_all_similarity(self):
# 执行完整的相似度计算流程
print("开始计算用户相似度...")
self.calc_common_rating_count()
self.calc_cosine_similarity()
self.calc_jaccard_similarity()
self.adaptive_weight_fusion()
self.shrinkage_process()
print("相似度计算完成！")
return self.final_sim_matrix

# KNN评分预测与推荐模块
class KNNRecommender:
def __init__(self, data_processor, user_modeler, sim_matrix, top_k):
self.data_processor = data_processor
self.user_modeler = user_modeler
self.sim_matrix = sim_matrix
self.top_k = top_k
self.n_users = data_processor.n_users
self.n_movies = data_processor.n_movies

def get_knn_neighbors(self, user_idx):
# 获取目标用户的Top-K近邻，返回(邻居索引, 相似度)
# 获取该用户与所有用户的相似度
user_sims = self.sim_matrix[user_idx]
# 按相似度降序排序，取Top-K

```

```

top_k_indices = np.argsort(user_sims)[::-1][:self.top_k]
top_k_sims = user_sims[top_k_indices]
return top_k_indices, top_k_sims

def predict_single_rating(self, user_idx, movie_idx):
    # 中心化加权平均, 预测目标用户对目标电影的评分
    rating_matrix = self.data_processor.rating_matrix
    user_mean = self.data_processor.user_mean_rating[user_idx]

    # 获取Top-K近邻
    neighbor_indices, neighbor_sims = self.get_knn_neighbors(user_idx)

    # 筛选出对该电影有评分的邻居
    rated_neighbor_mask = rating_matrix[neighbor_indices, movie_idx] > 0
    valid_neighbors = neighbor_indices[rated_neighbor_mask]
    valid_sims = neighbor_sims[rated_neighbor_mask]

    # 无有效邻居, 返回用户平均评分
    if len(valid_neighbors) == 0 or np.sum(np.abs(valid_sims)) == 0:
        return user_mean

    # 中心化加权平均计算预测评分
    neighbor_ratings = rating_matrix[valid_neighbors, movie_idx]
    neighbor_means = self.data_processor.user_mean_rating[valid_neighbors]

    # 公式:  $\text{pred} = \text{mean}_u + \frac{\sum(\text{sim} * (r_v - \text{mean}_v))}{\sum(|\text{sim}|)}$ 
    numerator = np.sum(valid_sims * (neighbor_ratings - neighbor_means))
    denominator = np.sum(np.abs(valid_sims))
    pred_rating = user_mean + numerator / denominator

    # 评分限制在1-5分范围内
    pred_rating = np.clip(pred_rating, 1.0, 5.0)
    return pred_rating

def recommend_for_user(self, user_idx, top_n):
    # 为目标用户生成Top-N推荐, 过滤已看过的电影
    rating_matrix = self.data_processor.rating_matrix
    # 筛选用户未评分的电影
    unrated_mask = rating_matrix[user_idx] == 0
    unrated_movie_indices = np.where(unrated_mask)[0]

    if len(unrated_movie_indices) == 0:
        print("该用户已对所有电影评分, 无推荐内容")
        return pd.DataFrame()

    # 批量预测所有未评分电影的评分
    pred_ratings = []
    for movie_idx in unrated_movie_indices:
        pred = self.predict_single_rating(user_idx, movie_idx)
        pred_ratings.append((movie_idx, pred))

    # 按预测评分降序排序, 取Top-N
    pred_ratings.sort(key=lambda x: x[1], reverse=True)
    top_n_preds = pred_ratings[:top_n]

    # 匹配电影信息, 生成推荐结果
    recommend_result = []
    for movie_idx, pred_rating in top_n_preds:
        movie_id, movie_name = self.data_processor.id_movie_map[movie_idx]
        recommend_result.append({
            "电影ID": movie_id,

```

```

        "电影名称": movie_name,
        "预测评分": round(pred_rating, 2)
    })

    return pd.DataFrame(recommend_result)

def get_user_history(self, user_idx):
    # 获取用户的历史评分记录，用于对比推荐结果
    rating_matrix = self.data_processor.rating_matrix
    rated_mask = rating_matrix[user_idx] > 0
    rated_movie_indices = np.where(rated_mask)[0]

    history = []
    for movie_idx in rated_movie_indices:
        movie_id, movie_name = self.data_processor.id_movie_map[movie_idx]
        rating = rating_matrix[user_idx, movie_idx]
        history.append({
            "电影ID": movie_id,
            "电影名称": movie_name,
            "用户评分": rating,
            "是否标记为喜欢": "是" if rating >= CONFIG["interest_threshold"] else "否"
        })

    return pd.DataFrame(history).sort_values("用户评分", ascending=False)

# 封装主程序
def main(ratings_path="ratings.csv", movies_path="movies.csv", target_user_idx=0):
    # 数据预处理
    data_processor = DataProcessor(ratings_path, movies_path)
    data_processor.load_and_process()

    # 用户建模
    user_modeler = UserModeler(data_processor, CONFIG["interest_threshold"])
    user_modeler.build_user_representation()

    # 相似度计算
    sim_calculator = SimilarityCalculator(user_modeler, CONFIG["alpha"], CONFIG["shrinkage_factor"])
    final_sim_matrix = sim_calculator.calculate_all_similarity()

    # 初始化推荐器
    recommender = KNNRecommender(data_processor, user_modeler, final_sim_matrix,
    CONFIG["top_k"])

    # 输出用户历史评分
    print(f"目标用户（索引={target_user_idx}）的历史评分记录\n")
    user_history = recommender.get_user_history(target_user_idx)
    print(user_history.to_string(index=False))

    # 生成并输出推荐结果
    print(f"\n为目标用户生成的Top-{CONFIG['top_n']}电影推荐\n")
    recommend_result = recommender.recommend_for_user(target_user_idx, CONFIG["top_n"])
    print(recommend_result.to_string(index=False))

    return data_processor, user_modeler, sim_calculator, recommender

# 程序入口
if __name__ == "__main__":
    # 运行主流程，通过修改target_user_idx指定不同用户
    main(ratings_path="ratings.csv", movies_path="movies.csv", target_user_idx=0)

```

(2) 整体思路

1.数据准备与预处理

首先需要将原始的评分数据 (`ratings.csv`) 和电影信息数据 (`movies.csv`) 加载进来。由于原始的用户 ID 和电影 ID 可能是不连续的, 或者是字符串形式, 不便于后续的矩阵运算, 因此需要建立 ID 到连续整数索引的映射。比如, 把原始的用户 ID 1、3、5…… 映射成 0、1、2……, 电影 ID 同理。接下来构建 “用户 - 电影评分矩阵”, 这是一个二维数组, 行对应用户, 列对应电影, 矩阵中的每个元素就是相应用户对相应电影的评分, 如果用户没给某部电影评过, 就填 0。同时, 还会计算每个用户的平均评分, 也就是把该用户所有有评分的电影的评分加起来除以数量, 这个平均评分后面会用到。

2.用户建模

为了更好地刻画用户, 系统采用了两种用户表示方式。

评分向量表示: 直接使用前面构建的 “用户 - 电影评分矩阵”, 每一行就是一个用户的评分向量, 完整保留了用户的评分行为信息。

兴趣集合表示: 设定一个 “兴趣阈值” (比如代码里默认是 4 分), 如果用户给某部电影的评分大于等于这个阈值, 并且确实有评分, 就认为用户 “喜欢” 这部电影。基于此, 构建一个 “兴趣二值矩阵”, 喜欢的位置填 1, 否则填 0; 同时, 为每个用户维护一个 “喜欢的电影索引集合”, 方便快速查看用户喜欢哪些电影; 还会统计每个用户一共喜欢多少部电影。

3.用户相似度计算

这是协同过滤的核心步骤, 目的是找出和目标用户 “品味相似” 的其他用户。系统综合了多种相似度计算方法, 并进行了优化。

计算共同评分数量: 先统计每两个用户之间共同评分过的电影有多少部。这里用了矩阵运算的技巧, 把评分矩阵转换成 “有评分填 1、没评分填 0” 的二值矩阵, 然后两个二值矩阵相乘, 结果矩阵中的元素就是对应的两个用户的共同评分数量。

共同子空间余弦相似度: 余弦相似度是衡量两个向量方向是否一致的指标, 但这里只在两个用户 “共同评分过的电影” 这个维度上计算, 避免了因为数据稀疏 (大部分用户只给少部分电影评过) 导致的相似度计算不准确。

兴趣 Jaccard 相似度: 基于前面的 “兴趣集合”, 计算两个用户喜欢的电影集合的 “交集” (都喜欢的电影数) 除以 “并集” (至少一个人喜欢的电影数), 这个比值越高, 说明两个用户的兴趣重叠度越高。

自适应加权融合: 把上面的余弦相似度和 Jaccard 相似度融合起来。融合的权重是自适应的: 如果两个用户的共同评分数量多, 就更信任余弦相似度, 给它更高的权重; 如果共同评分数量少, 就更信任 Jaccard 相似度, 给它更高的权重。权重的计算通过一个 “平衡参数 α ” 来调节。

Shrinkage 收缩处理: 这是为了降低 “小样本” 情况下的相似度估计误差。比如, 如果两个用户只有 1 部共同评分的电影, 哪怕算出的相似度很高, 也可能是偶然的。所以会用 “共同评分数量 / (共同评分数量 + 收缩系数)” 作为权重, 对融合后的相似度再做一次调整, 共同评分越少, 相似度被 “收缩” 得越厉害, 让结果更稳健。

4.评分预测与推荐生成

有了用户相似度之后, 就可以为目标用户预测他没看过的电影的评分, 并生成推荐了。

获取 Top-K 近邻: 对于目标用户, 从所有其他用户中选出和他相似度最高的 K 个用户 (代码里默认 $K=10$), 这就是 “近邻”。

中心化评分预测: 要预测目标用户对某部没看过的电影的评分, 采用 “中心化加权平均” 的方法。简单来说, 就是先找到近邻中给这部电影评过的人, 把他们的评分各自减去他们自己的平均评分 (这一步叫 “中心化”, 目的是消除不同用户的评分尺度差异, 比如有的用户习惯打高分, 有的习惯打低分), 然后乘以他们和目标用户的相似度, 把这些值加起来, 再除以相似度的绝对值之和, 最后加上目标用户自己的平均评分, 就得到了预测评分。如果近邻中没人给这部电影评过, 就直接用目标用户的平均评分作为预测。预测评分会被限制在 1-5 分之间。

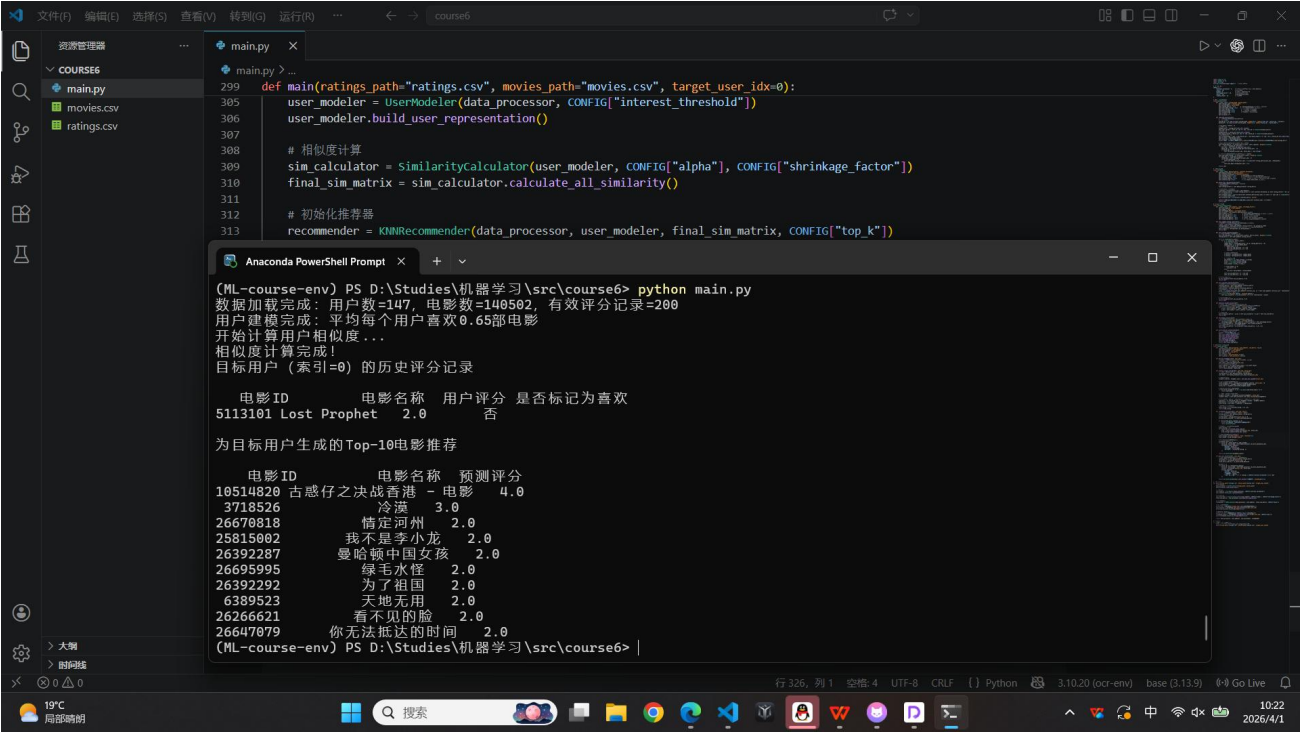
生成 Top-N 推荐: 首先筛选出目标用户没有评分过的所有电影, 然后对每一部都用上面的方法预测评分, 接着按预测评分从高到低排序, 取前 N 部 (代码里默认 $N=10$), 最后匹配上电影的 ID 和名称, 生成最终的推荐列表。同时, 系统也会输出目标用户的历史评分记录, 方便对比查看推荐是否符合用户的过往偏好。

5.总结

整个系统的核心逻辑就是 “物以类聚, 人以群分”: 通过用户的历史评分和兴趣找到相似的人, 再参考相似人的评分来预测目标用户的喜好, 最后推荐预测评分最高且没看过的电影。过程中用到的双用户表

示、自适应融合、Shrinkage 收缩、中心化预测等技巧，都是为了让推荐结果更准确、更稳健。

(3) 运行结果



```
def main(ratings_path="ratings.csv", movies_path="movies.csv", target_user_idx=0):
    user_modeler = UserModeler(data_processor, CONFIG["interest_threshold"])
    user_modeler.build_user_representation()

    # 相似度计算
    sim_calculator = SimilarityCalculator(user_modeler, CONFIG["alpha"], CONFIG["shrinkage_factor"])
    final_sim_matrix = sim_calculator.calculate_all_similarity()

    # 初始化推荐器
    recommender = KNNRecommender(data_processor, user_modeler, final_sim_matrix, CONFIG["top_k"])

    (ML-course-env) PS D:\Studies\机器学习\src\course6> python main.py
    数据加载完成: 用户数=147, 电影数=140502, 有效评分记录=200
    用户建模完成: 平均每个用户喜欢0.65部电影
    开始计算用户相似度...
    相似度计算完成!
    目标用户 (索引=0) 的历史评分记录

    电影ID      电影名称      用户评分      是否标记为喜欢
    5113101 Lost Prophet      2.0          否

    为目标用户生成的Top-10电影推荐

    电影ID      电影名称      预测评分
    10514820 古惑仔之决战香港 - 电影      4.0
    3718526  冷漠      3.0
    26670818 情定河州      2.0
    25815002 我不是李小龙      2.0
    26392287 曼哈顿中国女孩      2.0
    26695995 绿毛水怪      2.0
    26392292 为了祖国      2.0
    6389523  天地无用      2.0
    26266621 看不见的脸      2.0
    26647079 你无法抵达的时间      2.0
    (ML-course-env) PS D:\Studies\机器学习\src\course6>
```

4 实验总结与讨论

1. 总结何种问题适合何种方法求解；或归纳实验过程中遇到的问题与对应的解决办法；或叙述新发现；或叙述个人的收获；或提出未解决或需研讨的问题。

本次实验围绕影评数据分析与电影推荐核心任务，完整实现了基于用户的协同过滤推荐系统，全面达成了实验预设的学习与实践目标。

在实验完成度方面，我按照任务要求拆解了分阶段目标，完成了全流程的设计与实现。首先完成了数据集的加载与预处理，针对原始用户 ID、电影 ID 不连续的问题，构建了 ID 到连续整数索引的映射关系，成功搭建了用户 - 电影评分矩阵，为后续矩阵运算奠定了基础；其次完成了用户双维度建模，通过评分向量完整保留用户的评分行为信息，同时基于设定的兴趣阈值构建了用户兴趣集合与二值矩阵，从评分行为和偏好倾向两个维度完成了用户画像的刻画；随后实现了核心的相似度计算模块，完成了共同子空间余弦相似度、兴趣 Jaccard 相似度的计算，并设计了自适应加权融合策略，同时加入 Shrinkage 收缩处理优化小样本下的相似度估计偏差；最终基于 K 近邻算法，通过中心化加权平均方法实现了用户未观影电影的评分预测，完成了目标用户的 Top-N 电影推荐，同时输出了用户历史评分记录用于结果对比，完整验证了程序的正确性与可用性，完成了实验要求的全部任务。

在技术与理论学习方面，本次实验让我对协同过滤算法的核心逻辑有了系统性的深入理解，尤其是基于用户的协同过滤算法的完整落地流程，同时进一步加深了对 K 近邻算法在推荐场景中应用的认知，掌握了中心化加权平均方法消除用户评分尺度差异的核心原理。我也熟练掌握了余弦相似度、Jaccard 相似度两种核心相似度度量方法的适用场景与计算逻辑，学会了通过自适应加权融合两种算法的优势，以及通过收缩处理降低小样本误差的优化技巧。除此之外，我也提升了基于 NumPy 与 Pandas 的矩阵运算与数据处理能力，学会了通过矩阵运算替代循环操作提升代码运行效率的工程技巧。

在实验问题解决方面，我针对实验过程中遇到的多个核心问题完成了定位与优化。首先是数据集极度稀疏导致的相似度计算失真问题，本次实验所用数据集包含 147 个用户、140502 部电影，但有

效评分记录仅 200 条，矩阵稀疏度极高，绝大多数用户之间无共同评分电影，直接计算全量向量余弦相似度会出现结果严重偏差。针对该问题，我放弃了全量维度的相似度计算，改为仅在用户共同评分的电影子空间内计算余弦相似度，同时引入 Jaccard 相似度作为补充，通过自适应加权策略，在共同评分数量少时提升 Jaccard 相似度权重，共同评分数量多时提升余弦相似度权重，有效缓解了数据稀疏带来的计算偏差。其次是小样本共同评分导致的相似度偶然偏高问题，部分用户之间仅有 1-2 部共同评分电影，计算出的高相似度存在极大偶然性，会导致近邻选择错误。对此我引入了 Shrinkage 收缩处理，通过共同评分数量与收缩系数的比值作为权重修正融合相似度，共同评分数量越少，相似度收缩幅度越大，有效降低了小样本带来的估计方差，提升了相似度计算的稳健性。另外针对不同用户评分尺度差异带来的预测偏差问题，我采用了中心化的评分预测方法，先消除近邻用户自身的评分均值偏差，再进行相似度加权计算，最终叠加目标用户的平均评分得到预测结果，有效解决了用户评分习惯差异带来的系统误差。

本次实验也让我完成了从机器学习算法理论到工程落地的完整实践，不仅掌握了推荐系统的基础实现方法，更理解了推荐场景中数据稀疏、冷启动、评分偏差等核心问题的成因与基础解决方案，培养了面向实际问题拆解任务、设计方案、验证优化的全流程开发能力，积累了机器学习工程实践的宝贵经验。

2. 讨论最终得到的结果是否合理、是否达到题目要求； 分析可能的问题例如误差过大或效果不佳，分析产生的原因，探讨可能的解决方法。

从实验结果来看，本次实验完整实现了电影推荐的核心功能，输出了目标用户的历史评分记录与 Top-10 推荐列表，推荐逻辑完全符合基于用户的协同过滤算法的核心原理，达成了实验的基础要求。从结果合理性来看，目标用户仅对 1 部电影给出了 2.0 的评分，无标记为喜欢的电影，系统给出的推荐结果预测评分集中在 2.0-4.0 区间，与用户的平均评分水平匹配，同时完整过滤了用户已观看的电影，推荐结果具备基础的合理性。但受限于数据集质量与算法本身的特性，本次实验的推荐结果仍存在一定的优化空间，同时也暴露出一些值得深入探讨的问题。

首先是数据稀疏性对推荐效果的核心限制，这也是本次实验最突出的问题。本次实验所用数据集的有效评分仅 200 条，对应 147 个用户与 14 万余部电影，经计算矩阵稀疏度超过 99.99%，绝大多数用户仅存在 1-2 条评分记录，用户之间的共同评分极少，导致相似度计算缺乏足够的样本支撑，近邻用户的选择可靠性较低，最终使得评分预测的准确性与推荐的个性化程度不足。对于本次实验的目标用户，由于其仅存在 1 条评分记录，无法与其他用户形成有效的共同评分交集，系统无法精准刻画其观影偏好，最终的推荐结果更多依赖于用户的平均评分，个性化推荐能力大幅下降，这也凸显了协同过滤算法对数据量的强依赖特性。

其次是协同过滤算法固有的冷启动问题，在本次实验中表现十分明显。对于仅存在少量评分记录、甚至无评分记录的新用户，以及无评分记录的新电影，传统的协同过滤算法无法完成有效的相似度计算与偏好刻画，无法给出精准的个性化推荐。本次实验中大部分用户的喜欢电影数量不足 1 部，系统无法基于兴趣集合完成有效的相似度匹配，也正是冷启动问题的直接体现。

除此之外，本次实验的实现还存在一些工程与设计上的优化空间。在计算效率方面，本次实验的电影数量超 14 万，在批量预测电影评分时采用了循环计算的方式，虽然部分流程通过矩阵运算完成了优化，但面对超大规模数据集时，计算耗时会显著增加，内存占用也存在优化空间。在推荐评估方面，本次实验仅完成了推荐结果的输出，未引入定量的评估指标对推荐效果进行客观评价，无法精准量化评分预测的误差与推荐结果的质量，也无法完成超参数的系统性优化。同时，本次实验仅基于用户的评分数据进行推荐，未考虑用户评分的时间维度、电影的内容特征等信息，不仅无法解决冷启动问题，也使得推荐结果的多样性与可解释性不足。

针对以上问题，后续可以从多个方向进行优化改进。第一，针对数据稀疏问题优化算法模型，可以引入矩阵分解算法如 SVD、ALS 等，通过降维将用户与电影映射到低维隐空间，缓解数据稀疏带来的问题，也可以融合基于物品的协同过滤算法，与基于用户的协同过滤形成互补，提升推荐的稳健性。第二，针对冷启动问题引入多维度特征，融合电影的类型、导演、上映年份等内容特征，构建基于内容的推荐模块，与协同过滤算法形成混合推荐架构，同时可以引入用户的基础属性数据完善用户画像，进一步提升偏好刻画的精准度。第三，优化工程实现提升计算效率，通过稀疏矩阵存储评分数据降低内存占用，通过全量向量化运算替代循环操作减少计算耗时，同时可以通过预计算相似度矩阵的方式，避免推荐时的重复计算。第四，完善推荐评估体系与超参数优化，将数据集划分为训练集与测试集，引入 RMSE、MAE 指标评估评分预测精度，引入精准率、召回率、NDCG 等指标评估推荐质量，通过交叉验证的方式完成 top_k、平衡参数 alpha、收缩系数等超参数的最优解寻优，进一步提升推荐效果。第五，提升推荐的多样性与可解释性，在推荐时加入类型多样性约束，避免推荐内容同质化，同时在输出推荐结果时补充对应的推荐理由，提升用户对推荐结果的接受度。

从更宏观的视角来看，协同过滤作为经典的推荐算法，具备实现简单、可解释性强、无内容特征依赖的优势，在推荐系统发展过程中得到了广泛应用，但其本身也存在数据稀疏、冷启动、泛化能力弱等固有缺陷。在实际的工业级推荐系统中，通常不会单独使用协同过滤算法，而是采用混合推荐架构，融合协同过滤、基于内容的推荐、深度学习推荐模型等多种方法，同时结合用户行为、内容特征、场景信息等多维度数据，才能实现更精准、更稳健、更具多样性的推荐效果。本次实验也让我深刻认识到，机器学习算法的落地效果，不仅取决于算法本身的设计，更与数据集的质量、特征工程的设计、业务场景的适配密切相关，只有结合实际场景的特点进行针对性的优化，才能让算法发挥出最大的价值。